

Listas simplemente enlazadas en C

Cómo pensar la cabeza de la lista, cuándo devolver un puntero y cuándo usar `void`, y los moldes de inserción y eliminación para resolver ejercicios de parcial.

Este apunte organiza el razonamiento conceptual detrás de las listas simplemente enlazadas: qué es un nodo, por qué hay que "no perder la cabeza" al recorrer, cuándo una función debe devolver `Nodo *` y cuándo alcanza con `void`, y los cuatro moldes típicos para insertar y eliminar (por cabeza y dentro de la lista). Incluye el código de referencia y los diagramas de memoria paso a paso para cada caso.

1 Qué es lo que realmente tenemos que entender

Una lista simplemente enlazada es una sucesión de nodos. Cada nodo contiene:

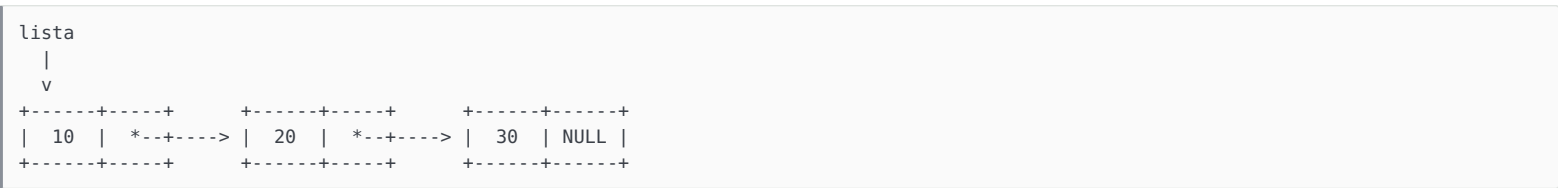
- información;
- un puntero al siguiente nodo.

La lista puede crecer o achicarse durante la ejecución del programa, porque cada nodo se reserva dinámicamente con `malloc` y, cuando deja de ser necesario, su memoria se libera con `free`.

Por ejemplo:

```
typedef struct Nodo {
    int dato;
    struct Nodo *sig;
} Nodo;
```

Gráficamente:



La variable:

```
Nodo *lista;
```

es especialmente importante porque contiene la dirección del primer nodo.

Ese primer nodo es la **cabeza** de la lista.

Si la lista está vacía:

```
lista = NULL;
```

2 La regla más importante: no perder la cabeza

IDEA CLAVE

Al recorrer una lista conviene mantener intacto el puntero que señala su comienzo y utilizar otro puntero para avanzar.

Por eso esto no suele ser una buena idea:

```
while (lista != NULL) {
    printf("%d\n", lista->dato);
    lista = lista->sig;
}
```

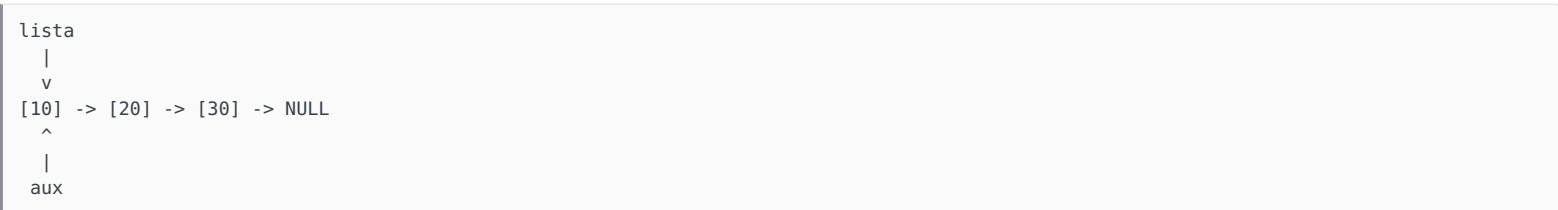
Dentro de la función puede no haber problema si `lista` es solamente una copia recibida por parámetro, pero conceptualmente conviene acostumbrarse a esto:

```
void mostrarLista(Nodo *lista) {
    Nodo *aux;

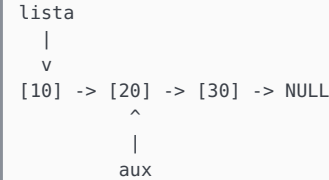
    aux = lista;

    while (aux != NULL) {
        printf("%d\n", aux->dato);
        aux = aux->sig;
    }
}
```

Tenemos entonces:



Y solamente `aux` avanza:



La cabeza sigue apuntando a `[10]`.

3 La pregunta clave antes de hacer cualquier función

Cuando tengas que insertar, eliminar o modificar algo de una lista, antes de escribir código hacete esta pregunta:

¿Esta operación puede cambiar cuál es el primer nodo de la lista?

Esta pregunta prácticamente determina cómo debería ser la función según el criterio que viene usando tu profesora.

Si NO modifica la cabeza

Normalmente:

```
void funcion(Nodo *lista);
```

Si SÍ modifica la cabeza

Normalmente:

```
Nodo *funcion(Nodo *lista);
```

y después:

```
lista = funcion(lista);
```

Este concepto aparece explícitamente en los apuntes tanto para inserción como para eliminación: cuando cambia la cabeza, la nueva dirección debe volver al programa principal.

Este es probablemente el concepto más importante para tus parciales.

4 ¿Por qué a veces usamos void y otras veces Nodo *?

Vamos a separar dos cosas.

Caso 1 — modificar el contenido de nodos

Supongamos:

```
void aumentar(Nodo *lista) {
    while (lista != NULL) {
        lista->dato++;
        lista = lista->sig;
    }
}
```

Esto funciona.

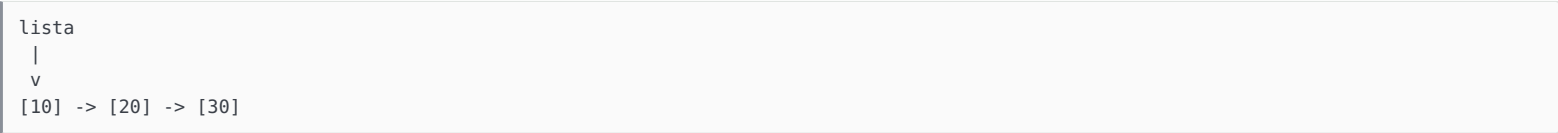
Aunque la función sea `void`, puede modificar:

- `lista->dato`
- `lista->sig`

porque tiene la dirección del nodo.

Caso 2 — modificar el puntero cabeza

Supongamos que tenemos:



y queremos eliminar 10. La nueva lista debe ser:

```
lista
|
v
[20] -> [30]
```

El valor de `lista` cambió. Por eso una función como:

```
Nodo *eliminarCabeza(Nodo *lista)
```

devuelve la nueva dirección:

```
lista = eliminarCabeza(lista);
```

5 Una forma de pensarlo que ayuda mucho

Considerá:

```
Nodo *lista;
```

como si fuera una variable normal. Por ejemplo:

```
int numero = 10;
```

Si una función tiene que calcular otro número:

```
int duplicar(int numero) {
    return numero * 2;
}
```

hacemos:

```
numero = duplicar(numero);
```

Con una lista ocurre algo parecido:

```
Nodo *eliminarCabeza(Nodo *lista) {
    ...
    return lista;
}
```

y:

```
lista = eliminarCabeza(lista);
```

La diferencia es que el valor que devolvemos no es un `int`, sino una dirección de memoria.

6 El criterio de tu profesora para insertar

Este punto es particularmente importante porque coincide con la corrección que te hizo.

En lugar de intentar resolver toda inserción con una única función complicada, hay que distinguir:

```
INSERTAR
|
+----> insertar modificando cabeza
|
+----> insertar dentro de la lista
```

El propio apunte hace esa separación: insertar por cabeza requiere devolver el nuevo cabezal; insertar en otra posición no modifica la dirección inicial.

No significa que cada ejercicio tenga que tener obligatoriamente ambas funciones.

Significa: usás la función que corresponda según lo que pide el problema.

7 Insertar por cabeza

Supongamos:

```
lista
|
v
[20] -> [30] -> NULL
```

Queremos insertar 10. Primero creamos:

```
aux
|
v
[10]
```

Después:

```
aux->sig = lista;
```

Tenemos:

```
aux
|
v
[10] -----+
              |
              v
lista --> [20] -> [30]
```

Finalmente:

```
lista = aux;
```

Resultado:

```
lista
|
v
[10] -> [20] -> [30]
```

Código:

```
Nodo *insertarCabeza(Nodo *lista, int dato) {
    Nodo *aux;

    aux = (Nodo *)malloc(sizeof(Nodo));

    aux->dato = dato;

    aux->sig = lista;
    lista = aux;

    return lista;
}
```

Y desde main:

```
lista = insertarCabeza(lista, 10);
```

Fijate que `Nodo *` tiene sentido porque cambió el cabezal.

8 Los dos enlaces de una inserción

Cuando insertás en el medio de una lista, hay algo que quiero que memorices conceptualmente, no de memoria como código.

Supongamos:

```
[10] -> [30]
```

queremos insertar:

```
[20]
```

Tenemos:

```
registro
|
v
[10] -----> [30]
```

Primero hacemos:

```
aux->sig = registro->sig;
```

Entonces:



Después:

```
registro->sig = aux;
```

Resultado:

```
[10] -> [20] -> [30]
```

CÓMO LO RESUME LA PROFESORA

Reservar memoria, enlazar el nodo nuevo con la lista y después enlazar la lista con el nodo nuevo.

9 Insertar dentro de la lista

Supongamos que queremos insertar después de determinado nodo.

Podríamos tener:

```
void insertarDespues(Nodo *registro, int dato) {
    Nodo *aux;

    aux = (Nodo *)malloc(sizeof(Nodo));

    aux->dato = dato;

    aux->sig = registro->sig;
    registro->sig = aux;
}
```

Notá que devuelve `void`. ¿Por qué? Porque esto:



sigue siendo igual. No cambiamos `lista`. Solamente modificamos enlaces dentro de la lista.

10 Ejemplo con condición de inserción

Supongamos que queremos insertar 25 después del nodo cuyo dato sea 20.

```
void insertar(Nodo *lista, int dato) {
    Nodo *registro;
    Nodo *aux;

    registro = lista;

    while (registro != NULL) {

        if (registro->dato == 20) {

            aux = (Nodo *)malloc(sizeof(Nodo));

            aux->dato = dato;

            aux->sig = registro->sig;
            registro->sig = aux;

            return;
        }

        registro = registro->sig;
    }
}
```

Acá tenemos:

```
lista
 |
 v
[10] -> [20] -> [30]
      ^
      |
    registro
```

y terminamos:

```
[10] -> [20] -> [25] -> [30]
```

11 Eliminar: el mismo razonamiento

Con eliminación aplicamos exactamente la misma división:

```
ELIMINAR
|
+----> eliminar cabeza
|
+----> eliminar dentro de la lista
```

El material también lo separa de esta manera. Además, para eliminar un nodo señala tres ideas esenciales: guardar la referencia necesaria, liberar memoria y volver a enlazar la lista.

12 Eliminar la cabeza

Tenemos:

```
lista
|
v
[10] -> [20] -> [30]
```

Queremos eliminar 10. No podemos hacer directamente:

```
free(lista);
lista = lista->sig;
```

porque después de `free(lista)` ya no debemos acceder al nodo eliminado.

Primero debemos guardar el siguiente.

```
aux = lista->sig;
```

Tenemos:

```
lista
|
v
[10] -> [20] -> [30]
      ^
      |
      aux
```

Después:

```
free(lista);
```

y finalmente:

```
lista = aux;
```

Código:

```
Nodo *eliminarCabeza(Nodo *lista) {
    Nodo *aux;

    if (lista != NULL) {
        aux = lista->sig;
        free(lista);
        lista = aux;
    }

    return lista;
}
```

Desde main:

```
lista = eliminarCabeza(lista);
```

Los pasos coinciden con los del apunte de la profesora: guardar `prim->sig`, liberar `prim` y actualizar el cabezal.

13 Eliminar un nodo del medio

Este caso cambia bastante. Tenemos:

```
[10] -> [20] -> [30]
  ^
  |
lista
```

Queremos eliminar 20. Para eliminar 20, necesitamos posicionarnos en [10] . No en 20.

¿Por qué? Porque necesitamos modificar registro->sig para que deje de apuntar a 20 y pase a apuntar a 30.

```
registro
  |
  v
[10] -> [20] -> [30]
```

Guardamos:

```
aux = registro->sig;
```

Entonces aux es el nodo a eliminar:

```
registro
  |
  v
[10] -> [20] -> [30]
          ^
          |
          aux
```

Después:

```
registro->sig = aux->sig;
```

Queda:

```
[10] -----> [30]
           [20]
           ^
           |
           aux
```

Finalmente:

```
free(aux);
```

Resultado:

```
[10] -> [30]
```

14 Función para eliminar dentro de la lista

Ejemplo: eliminar nodos cuyo dato sea menor a 30, suponiendo que el cabezal se tratará aparte.

```
void eliminarResto(Nodo *lista) {
    Nodo *registro;
    Nodo *aux;

    registro = lista;

    while (registro != NULL && registro->sig != NULL) {

        if (registro->sig->dato < 30) {

            aux = registro->sig;
            registro->sig = aux->sig;
            free(aux);

        } else {

            registro = registro->sig;

        }

    }
}
```

Este patrón es extremadamente importante. Observá que la condición mira:

```
registro->sig->dato
```

y no:

```
registro->dato
```

porque `registro` es el nodo anterior al que podemos eliminar.

15 Una regla MUY importante al eliminar

Supongamos:

```
[50] -> [10] -> [20] -> [60]
```

y queremos eliminar los menores a 30. Estamos acá:

```
registro
|
v
[50] -> [10] -> [20] -> [60]
```

Eliminamos 10. Ahora:

```
registro
|
v
[50] -> [20] -> [60]
```

NO AVANZAMOS REGISTRO

Porque el nuevo `registro->sig` es 20 y también puede tener que eliminarse.

Esto está remarcado explícitamente en el material de la profesora: **cuando se elimina un nodo no se avanza; cuando no se elimina, recién entonces se avanza.**

Por eso:

```
if (condicion) {
    // eliminar
}
else {
    registro = registro->sig;
}
```

No:

```
if (condicion) {
    // eliminar
}

registro = registro->sig;
```

Ese pequeño detalle evita muchísimos errores.

16 Caso completo según el criterio de tu profesora

Supongamos el ejercicio: *eliminar todos los nodos cuyo dato sea menor a 30*.

Puede ocurrir esto:

```
[10] -> [20] -> [40] -> [15] -> [50]
```

Tenemos un problema: los primeros nodos también cumplen la condición. Entonces lo separamos en dos funciones.

Primera función: eliminar por cabeza

```
Nodo *eliminarCabezaMenores30(Nodo *lista) {
    Nodo *aux;

    while (lista != NULL && lista->dato < 30) {

        aux = lista->sig;
        free(lista);
        lista = aux;
    }

    return lista;
}
```

¿Por qué `while` y no `if`? Porque puede haber:

```
[10] -> [15] -> [20] -> [50]
```

y hay que eliminar varios nodos consecutivos del principio. Después queda:

```
[50]
 ^
 |
lista
```

Segunda función: eliminar del resto

```
void eliminarRestoMenores30(Nodo *lista) {
    Nodo *registro;
    Nodo *aux;

    registro = lista;

    while (registro != NULL && registro->sig != NULL) {

        if (registro->sig->dato < 30) {

            aux = registro->sig;
            registro->sig = aux->sig;
            free(aux);

        } else {

            registro = registro->sig;
        }
    }
}
```

Y en main:

```
lista = eliminarCabezaMenores30(lista);

eliminarRestoMenores30(lista);
```

Esto representa muy bien el criterio que te viene pidiendo la profesora.

17 La diferencia que quiero que memorices

```

¿MODIFICO LA CABEZA?
  |
+---+---+
  |       |
  SÍ       NO
  |       |
  v       v
Nodo *    void
return    no return
lista     necesario

```

Ejemplo:

```
lista = eliminarCabeza(lista);
```

contra:

```
eliminarResto(lista);
```

18 ¿Entonces nunca usamos Nodo **lista?

No. En C perfectamente podríamos hacer:

```
void eliminar(Nodo **lista)
```

y cambiar directamente:

```
*lista
```

Pero eso agrega un nivel más de indirección.

Según el criterio con el que viene trabajando tu profesora, no hace falta resolver estos ejercicios con doble puntero. La alternativa que está usando es mucho más clara para esta etapa:

```
Nodo *eliminarCabeza(Nodo *lista);
```

y:

```
lista = eliminarCabeza(lista);
```

Para el resto:

```
void eliminarResto(Nodo *lista);
```

Esto además hace explícito cuándo se está modificando la cabeza.

19 Ojo: void no significa "no modifica nada"

Este es otro concepto que suele generar confusión. Una función:

```
void funcion(Nodo *lista)
```

perfectamente puede modificar la lista. Por ejemplo:

```
void cambiar(Nodo *lista) {
    lista->dato = 100;
}
```

modifica el nodo. También:

```
void cortar(Nodo *lista) {
    lista->sig = NULL;
}
```

modifica un enlace.

Lo que no puede hacer de esta forma es comunicarle al main:

```
lista = otroNodo;
```

porque el parámetro `lista` de la función es una copia del puntero.

20 Esto explica algo muy importante sobre los punteros

Supongamos:

```
void prueba(Nodo *lista) {  
    lista = lista->sig;  
}
```

Después:

```
prueba(lista);
```

el `lista` del main no cambió.

Pero:

```
void prueba(Nodo *lista) {  
    lista->dato = 100;  
}
```

sí cambia el contenido del nodo.

¿Por qué? Porque una cosa es cambiar `lista` y otra cosa es cambiar `*lista` o `lista->dato`.

Podés pensarlo así:

```
MAIN  
  
lista -----> nodo
```

Cuando llamamos:

```
funcion(lista);
```

tenemos:

MAIN		FUNCIÓN
lista -----+		parametro
+----->		nodo

Los dos punteros tienen la misma dirección. Por eso ambos pueden modificar el nodo. Pero son dos variables puntero diferentes.

Si dentro de la función hacemos:

```
parametro = parametro->sig;
```

solamente cambió `parametro`.

21 Mostrar una lista

Una función de mostrar debería ser `void` :

```
void mostrarLista(Nodo *lista) {
    Nodo *registro;

    registro = lista;

    while (registro != NULL) {

        printf("%d\n", registro->dato);

        registro = registro->sig;
    }
}
```

Porque no genera una nueva cabeza, no necesita devolver una lista, solamente recorre e imprime.

22 Contar elementos

Por ejemplo:

```
int contar(Nodo *lista) {
    Nodo *registro;
    int contador = 0;

    registro = lista;

    while (registro != NULL) {

        contador++;

        registro = registro->sig;
    }

    return contador;
}
```

Acá no tiene sentido `void` , porque queremos obtener un resultado:

```
cantidad = contar(lista);
```

23 Calcular un promedio

Mismo razonamiento:

```
float promedio(Nodo *lista) {
    Nodo *registro;
    int cantidad = 0;
    float acumulador = 0;

    registro = lista;

    while (registro != NULL) {

        acumulador += registro->dato;
        cantidad++;

        registro = registro->sig;
    }

    if (cantidad != 0)
        return acumulador / cantidad;

    return 0;
}
```

Entonces:

```
printf("%.2f", promedio(lista));
```

24 Cuándo usar cada tipo de función

Esta tabla te puede servir bastante para estudiar:

La función...	Tipo habitual
Muestra una lista	<code>void</code>
Recorre y modifica datos internos	<code>void</code>
Inserta sin cambiar cabeza	<code>void</code>
Elimina sin cambiar cabeza	<code>void</code>
Cuenta elementos	<code>int</code>
Calcula promedio	<code>float</code>
Busca y devuelve un nodo	<code>Nodo *</code>
Genera una nueva lista	<code>Nodo *</code>
Inserta cambiando la cabeza	<code>Nodo *</code>
Elimina cambiando la cabeza	<code>Nodo *</code>

Pero no memorices solamente la tabla. La idea es:

El tipo de la función representa qué información necesita devolver al que la llamó.

25 Generar una nueva lista

Esto es un caso diferente de modificar una lista existente. Supongamos: *generar una nueva lista con los empleados de entre 30 y 45 años*.

Podríamos hacer:

```
Empleado *generarLista(Empleado *lista) {
    Empleado *nueva = NULL;

    // recorrer lista
    // reservar nodos
    // agregarlos a nueva

    return nueva;
}
```

Entonces en main:

```
listaNueva = generarLista(lista);
```

Acá tiene mucho sentido `Empleado *`, porque la función está construyendo una nueva cabeza de lista y necesita devolvértela.

26 Tres punteros que vas a ver constantemente

Cuando trabajes con listas, probablemente aparezcan mucho estos nombres:

- `lista`
- `registro`
- `aux`

Podés asociarlos mentalmente así:

lista — Cabeza

```
lista
|
v
[10] -> [20] -> [30]
```

Idealmente no la usamos para recorrer.

registro — El que recorre

```
lista
|
v
[10] -> [20] -> [30]
      ^
      |
    registro
```

aux — Un puntero temporal

Puede servir para:

- crear un nodo;
- guardar un nodo antes de eliminarlo;
- conservar una dirección.

Por ejemplo:

```
aux = registro->sig;
```

27 Patrón mental para INSERTAR

Cuando veas **insertar**, pensá:

1. malloc
2. cargar datos
3. enlazar nuevo nodo con lista
4. enlazar lista con nuevo nodo

Ejemplo:

```
aux = malloc(sizeof(Nodo));

aux->dato = dato;

aux->sig = registro->sig;
registro->sig = aux;
```

OJO CON EL ORDEN

Fijate especialmente en el orden: `aux->sig = registro->sig;` antes que `registro->sig = aux;` , porque si cambiás primero `registro->sig` , podrías perder la referencia al resto de la lista.

28 Patrón mental para ELIMINAR

Cuando veas **eliminar**, pensá:

1. encontrar nodo anterior
2. guardar nodo que voy a borrar
3. saltarlo en la lista
4. free

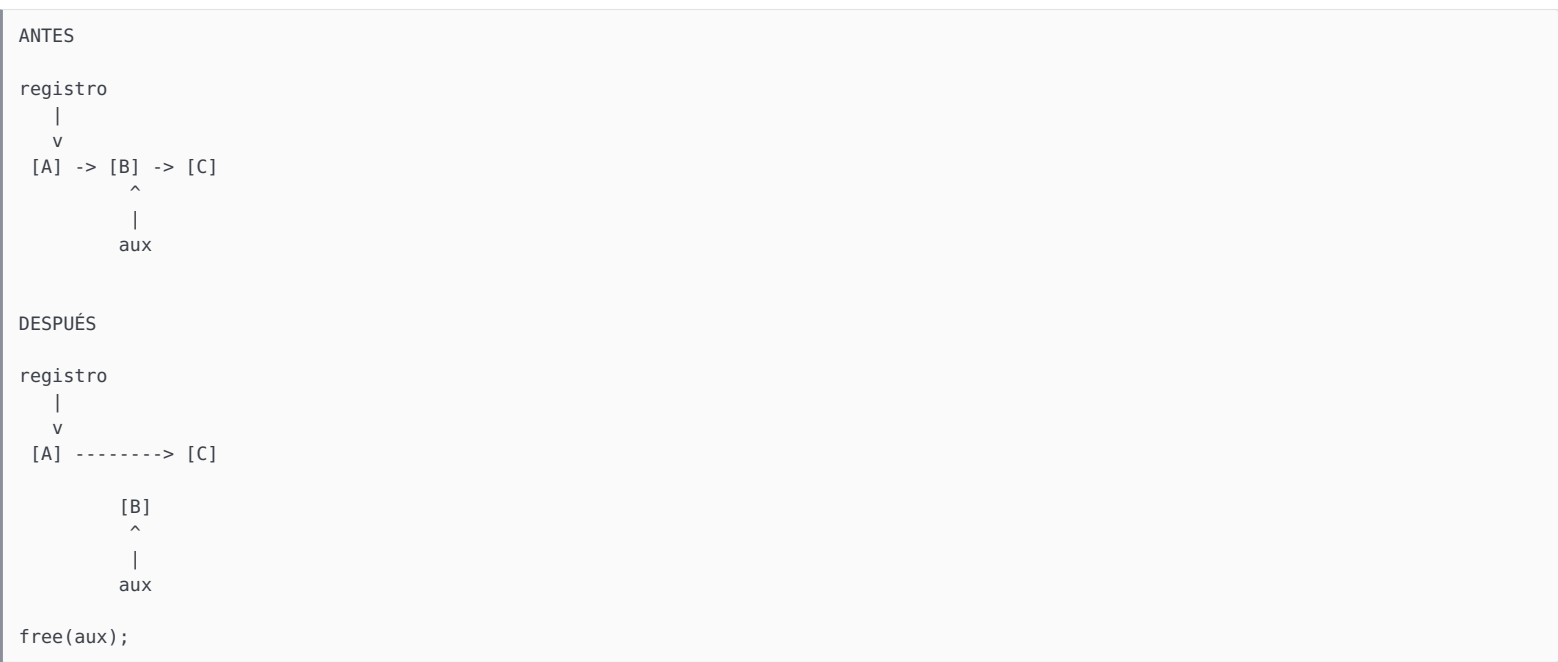
Ejemplo:

```
aux = registro->sig;

registro->sig = aux->sig;

free(aux);
```

Gráficamente:



29 Los errores que más deberías vigilar

Al resolver un ejercicio de listas, revisaría especialmente estas situaciones:

```
registro = registro->sig;
```

Preguntate siempre: **¿tengo que avanzar en este momento?** En eliminación consecutiva, probablemente no.

También revisá:

```
registro->sig
```

antes de hacer:

```
registro->sig->dato
```

porque si `registro->sig == NULL`, intentar acceder a `registro->sig->dato` provoca un acceso inválido.

Por eso aparecen condiciones como:

```
while (registro != NULL && registro->sig != NULL)
```

30 Checklist para resolver un ejercicio en parcial

Cuando te den algo como: *eliminar los candidatos con menos de 30 votos*, no empieces escribiendo código inmediatamente. Primero razoná:

1. ¿Trabajo sobre la lista original o genero otra?

En este ejemplo, lista original.

2. ¿Puedo eliminar el primer nodo?

Sí. Entonces necesito contemplar modificación del cabezal.

3. ¿Puede haber varios nodos consecutivos al principio?

Sí. Entonces probablemente `while (...)` en la función de cabeza.

4. ¿También puede haber nodos a eliminar en el medio?

Sí. Entonces necesito otra función para el resto.

Resultado conceptual:

```
Partido *eliminarCabeza(Partido *lista);  
  
void eliminarResto(Partido *lista);
```

Y:

```
lista = eliminarCabeza(lista);  
eliminarResto(lista);
```

Antes de escribir una sola línea interna ya resolviste gran parte del problema.

31 La idea central para llevarte

Si tuviera que condensar todo este tema en una sola regla, sería:

Cuando trabajes con listas, antes de pensar en el `while`, pensá qué puntero necesitás conservar y qué puntero necesitás modificar.

Después:

```
Si cambia la cabeza  
    ↓  
devuelvo la nueva cabeza  
    ↓  
Nodo *funcion(...)  
  
Si no cambia la cabeza  
    ↓  
puedo trabajar sobre sus nodos/enlaces  
    ↓  
void funcion(...)
```

Y particularmente para insertar y eliminar, con el criterio que te viene marcando tu profesora, pensaría siempre en estos cuatro moldes:

```
Nodo *insertarCabeza(...);  
  
void insertarResto(...);  
  
Nodo *eliminarCabeza(...);  
  
void eliminarResto(...);
```

PARA LLEVARTE

No significa que un ejercicio vaya a necesitar las cuatro. Significa que, cuando el enunciado te obligue a insertar o eliminar, primero identificás cuál de esos casos corresponde.

Ese razonamiento es bastante más importante que aprender de memoria una función concreta.